

HAFTA 6

Deney 1: Trafik Lambası Simülasyonu

Amaç: Kırmızı, sarı ve yeşil LED'ler kullanarak basit bir trafik lambası sistemi oluşturmak.

Malzemeler:

- 1 adet kırmızı LED
- 1 adet sarı LED
- 1 adet yeşil LED
- 3 adet 220Ω direnç
- Arduino UNO
- Breadboard
- Jumper kablolar

Deneyin Mantığı:

Trafik lambası belirli bir sırayla çalışır:

- Kırmızı LED yanar
- Sarı LED yanar
- Yeşil LED yanar
- Sistem tekrar başa döner

Bu deneyde öğrenciler zamanlama mantığını ve dijital çıkış kontrolünü öğrenir.

Devre Bağlantıları:

- Kırmızı LED → Pin 2
- Sarı LED → Pin 3
- Yeşil LED → Pin 4
- Her LED'in kısa bacağı → GND
- Her LED'in uzun bacağı → 220Ω direnç üzerinden Arduino pinine bağlanır

Arduino Kodu:

```
// LED pinleri tanımlanır
int kirmiziLed = 2;
int sariLed = 3;
int yesilLed = 4;

void setup() {
  // LED pinleri çıkış olarak ayarlanır
  pinMode(kirmiziLed, OUTPUT);
  pinMode(sariLed, OUTPUT);
  pinMode(yesilLed, OUTPUT);
}

void loop() {
  // Kırmızı LED yanar
```

```
digitalWrite(kirmiziLed, HIGH);
digitalWrite(sariLed, LOW);
digitalWrite(yesilLed, LOW);

// 3 saniye beklenir
delay(3000);

// Sarı LED yanar
digitalWrite(kirmiziLed, LOW);
digitalWrite(sariLed, HIGH);
digitalWrite(yesilLed, LOW);

// 1 saniye beklenir
delay(1000);

// Yeşil LED yanar
digitalWrite(kirmiziLed, LOW);
digitalWrite(sariLed, LOW);
digitalWrite(yesilLed, HIGH);

// 3 saniye beklenir
delay(3000);

// Sarı LED tekrar yanar
digitalWrite(kirmiziLed, LOW);
digitalWrite(sariLed, HIGH);
digitalWrite(yesilLed, LOW);

// 1 saniye beklenir
delay(1000);
}
```

Deney 2: Ultrasonik Park Sensörü

Amaç: HC-SR04 ultrasonik sensör ile mesafe ölçmek ve ölçülen mesafeye göre servo motorun açısını değiştirmek.

Malzemeler:

- 1 adet HC-SR04 Ultrasonik Sensör
- 1 adet SG90 Servo Motor
- Arduino UNO
- Breadboard
- Jumper kablolar

Deneyin Mantığı:

HC-SR04 sensörü ses dalgaları göndererek cismin uzaklığını ölçer.

Sensör:

- Trig pini ile ses dalgası gönderir.
- Echo pini ile geri dönen sinyali algılar.
- Arduino bu süreyi ölçerek mesafeyi hesaplar.

Bu deneyde servo motor mesafeye göre farklı açılara döner:

- Uzak mesafe $\rightarrow 0^\circ$
- Orta mesafe $\rightarrow 90^\circ$
- Çok yakın mesafe $\rightarrow 180^\circ$

Devre Bağlantıları:

HC-SR04:

- VCC $\rightarrow$ 5V
- GND $\rightarrow$ GND
- Trig $\rightarrow$ Pin 6
- Echo $\rightarrow$ Pin 7

Servo Motor:

- Kırmızı kablo $\rightarrow$ 5V
- Kahverengi kablo $\rightarrow$ GND
- Turuncu kablo $\rightarrow$ Pin 9

Arduino Kodu:

```
// Servo kütüphanesi eklenir
#include <Servo.h>

// Servo nesnesi oluşturulur
Servo parkServo;
```

```
// Ultrasonik sensör pinleri
int trigPin = 6;
int echoPin = 7;

// Mesafe ölçümü için değişkenler
long sure;
int mesafe;

void setup() {
  // Trig pini çıkış olarak ayarlanır
  pinMode(trigPin, OUTPUT);

  // Echo pini giriş olarak ayarlanır
  pinMode(echoPin, INPUT);

  // Servo motor 9 numaralı pine bağlanır
  parkServo.attach(9);

  // Seri haberleşme başlatılır
  Serial.begin(9600);
}

void loop() {
  // Trig pini önce LOW yapılır
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  // Trig pininden kısa bir HIGH sinyali gönderilir
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Echo pininden dönen sinyalin süresi ölçülür
  sure = pulseIn(echoPin, HIGH);

  // Mesafe cm cinsinden hesaplanır
  mesafe = sure * 0.034 / 2;

  // Ölçülen mesafe seri ekrana yazdırılır
  Serial.print("Mesafe: ");
  Serial.print(mesafe);
  Serial.println(" cm");

  // Eğer cisim uzaktaysa servo 0 dereceye döner
  if (mesafe > 30) {
    parkServo.write(0);
  }

  // Eğer cisim orta mesafedeyse servo 90 dereceye döner
  else if (mesafe > 10 && mesafe <= 30) {
```

```
    parkServo.write(90);  
}  
  
// Eğer cisim çok yakındaysa servo 180 dereceye döner  
else {  
    parkServo.write(180);  
}  
  
// Ölçümler arasında kısa bekleme  
delay(300);  
}
```

Deney 3: Joystick Butonu ile Akıllı LED Sistemi

Amaç: Joystick modülünün X-Y eksenlerini ve butonunu kullanarak LED'leri farklı şekillerde kontrol etmek. Dijital giriş, analog giriş, dijital çıkış, dizi kullanımı ve debounce mantığını birlikte uygulamak.

Malzemeler:

- 1 adet Joystick modülü
- 4 adet LED
- 4 adet 220 Ω direnç
- Arduino UNO
- Breadboard
- Jumper kablolar

Deneyin Mantığı:

Joystick modülü üzerinde X eksen, Y eksen ve buton çıkışı bulunur.

Bu deneyde:

- Joystick sağa itilirse aktif LED sağa doğru ilerler.
- Joystick sola itilirse aktif LED sola doğru ilerler.
- Joystick yukarı itilirse tüm LED'ler yanar.
- Joystick aşağı itilirse tüm LED'ler söner.
- Joystick butonuna basılırsa aktif LED bir sonraki LED'e geçer.
- Butondaki titreşimleri engellemek için debounce mantığı kullanılır.

Devre Bağlantıları:

Joystick:

- VCC $\rightarrow$ 5V
- GND $\rightarrow$ GND
- VRx $\rightarrow$ A0
- VRy $\rightarrow$ A1
- SW $\rightarrow$ Pin 2

LED'ler:

- LED 1 $\rightarrow$ Pin 3
- LED 2 $\rightarrow$ Pin 4
- LED 3 $\rightarrow$ Pin 5
- LED 4 $\rightarrow$ Pin 6
- Tüm LED'lerin kısa bacakları $\rightarrow$ GND
- LED'lerin uzun bacakları $\rightarrow$ 220 Ω direnç üzerinden Arduino pinlerine bağlanır

Arduino Kodu:

```

// -----
// JOYSTICK İLE GELİŞMİŞ LED KONTROL SİSTEMİ
// -----
// Sağ/Sol   → LED değiştirir
// Yukarı    → Tüm LED'ler yanar (mod)
// Aşağı     → Tüm LED'ler söner (mod)
// Buton     → Bir sonraki LED'e geçer
// -----

// Joystick analog pinleri
int xPin = A0;
int yPin = A1;

// Joystick buton pini
int buttonPin = 2;

// LED pinleri (dizi kullanımı)
int leds[] = {3, 4, 5, 6};

// Aktif LED'in indeksini tutar
int aktifLed = 0;

// Sistem modları
bool tumLedlerAcik = false; // Tüm LED'ler açık mı?
bool sistemKapali = false;  // Tüm LED'ler kapalı mı?

// Butonun önceki durumu (INPUT_PULLUP olduğu için HIGH başlar)
int oncekiButonDurumu = HIGH;

// Debounce için zaman değişkenleri
unsigned long sonDegisimZamani = 0;
unsigned long debounceSuresi = 50;

// Joystick hareketlerinin çok hızlı tekrar etmesini engellemek için
unsigned long sonHareketZamani = 0;
unsigned long hareketBeklemeSuresi = 300;

void setup() {

    // Tüm LED pinleri çıkış olarak ayarlanır
    for (int i = 0; i < 4; i++) {
        pinMode(leds[i], OUTPUT);
    }

    // Joystick butonu INPUT_PULLUP olarak ayarlanır
    // (harici direnç gerektirmez)
    pinMode(buttonPin, INPUT_PULLUP);
}

void loop() {

```

```

// Joystick X ve Y eksen değerleri okunur (0 - 1023 arası)
int xVal = analogRead(xPin);
int yVal = analogRead(yPin);

// Buton durumu okunur
int butonDurumu = digitalRead(buttonPin);

// Buton durumunda değişiklik varsa zaman kaydedilir
if (butonDurumu != oncekiButonDurumu) {
    sonDegisimZamani = millis();
}

// Debounce kontrolü (titreşim engelleme)
if ((millis() - sonDegisimZamani) > debounceSuresi) {

    // Butona basılma anı yakalanır (HIGH → LOW)
    if (butonDurumu == LOW && oncekiButonDurumu == HIGH) {

        // Bir sonraki LED'e geç
        aktifLed++;

        // 4 LED varsa başa dön
        if (aktifLed > 3) {
            aktifLed = 0;
        }

        // Modlardan çık (normal moda dön)
        tumLedlerAcik = false;
        sistemKapali = false;
    }
}

// Joystick hareketleri belirli aralıklarla okunur
if ((millis() - sonHareketZamani) > hareketBeklemeSuresi) {

    // Sağa itilirse → LED ileri gider
    if (xVal > 800) {

        aktifLed++;

        if (aktifLed > 3) {
            aktifLed = 0;
        }

        tumLedlerAcik = false;
        sistemKapali = false;

        sonHareketZamani = millis();
    }

    // Sola itilirse → LED geri gider

```

```

else if (xVal < 200) {

    aktifLed--;

    if (aktifLed < 0) {
        aktifLed = 3;
    }

    tumLedlerAcik = false;
    sistemKapali = false;

    sonHareketZamani = millis();
}

// Yukarı itilirse → tüm LED'ler yanar (mod)
else if (yVal > 800) {

    tumLedlerAcik = true;
    sistemKapali = false;

    sonHareketZamani = millis();
}

// Aşağı itilirse → tüm LED'ler söner (mod)
else if (yVal < 200) {

    tumLedlerAcik = false;
    sistemKapali = true;

    sonHareketZamani = millis();
}
}

// ----- LED ÇIKIŞ KONTROLÜ -----

// Eğer tüm LED'ler açık moddaysa
if (tumLedlerAcik == true) {

    for (int i = 0; i < 4; i++) {
        digitalWrite(leds[i], HIGH);
    }
}

// Eğer sistem kapalı moddaysa
else if (sistemKapali == true) {

    for (int i = 0; i < 4; i++) {
        digitalWrite(leds[i], LOW);
    }
}
}

```

```
// Normal mod: sadece aktif LED yanar
else {

    // Önce tüm LED'leri kapat
    for (int i = 0; i < 4; i++) {
        digitalWrite(leds[i], LOW);
    }

    // Sadece aktif LED'i yak
    digitalWrite(leds[aktifLed], HIGH);
}

// Butonun mevcut durumu saklanır
oncekiButonDurumu = butonDurumu;
}
```